MoneyMochi

Build Your Own Stock Dashboard App

A friendly, step-by-step guide - from the single HTML file to a live app

Second edition - now with watchlist, portfolio summary, news, alerts and backend

WHAT THIS IS

This guide turns the MoneyMochi dashboard you already have into a real, daily-updating app. It covers getting your free API keys, running the dashboard, the price-history chart, price alerts, and the full path to a hosted app with a backend and scheduled updates. No prior backend experience assumed - where helpful, it points out what you can hand to Claude Code to build for you.

PLEASE READ

Educational tool only. Nothing here is financial advice or a recommendation to buy or sell. Market data shown in your app is for information; always verify on your brokerage.

Contents

FEATURE RECAP

What your app already does today: search any stock; full written analysis for Alphabet, Microsoft, Meta and Amazon; live price, valuation and financial-health metrics for any other ticker; a one-year price-history chart; latest news per stock; support and resistance zones with rough entry/exit guidance; price alerts; and a Watchlist home screen with a portfolio summary and an 'entry watch' column showing which holdings are near their support zone.

Part 1 - Get your free API keys

An API key is just a password that lets your app ask a data company for stock prices. You need two free keys. Both take a couple of minutes and neither asks for a credit card.

Key A - Finnhub (live prices, fundamentals, and search)

1. Open **finnhub.io/register** in your browser.
2. Sign up with your email and a password (the free plan is selected by default - no card needed).
3. Check your inbox and click the verification link, then log in.
4. You land on the dashboard. Your **API key** is shown as a long string of letters and numbers - copy it.
5. Keep it somewhere safe. This single key powers live prices, the financial snapshot, the search box, and the price alerts.

> **GOOD TO KNOW**
> Finnhub free tier: about 60 requests per minute, covers US and most major global stocks. US quotes are near real-time; some data is delayed. That is plenty for a personal dashboard.

Key B - Twelve Data (the price-history chart)

1. Open **twelvedata.com** and click **Sign up** (Free plan).
2. Confirm your email and log in.
3. In the dashboard, find **API Keys** and copy your key.
4. This one is only used to draw the one-year price-history chart.

> **GOOD TO KNOW**
> Twelve Data free tier: about 800 requests per day and 8 per minute - more than enough to load history charts as you browse. This key is optional; the rest of the dashboard works without it.

Part 2 - Run the dashboard and switch on live data

You already have the file **moneymochi-multistock.html**. It is a complete app in one file - no installation needed.

Open it

- Double-click the HTML file - it opens in your web browser. That's it.
- Or host it for free (so you can open it from your phone) - see Part 5.

Add your keys (two ways)

Quick way (per session): click the **Live data** button at the top right, paste your Finnhub key (and optionally your Twelve Data key), and press **Save keys**. Live prices, search-any-stock, history, and alerts all switch on.

Permanent way (recommended once you download it): open the HTML file in any text editor and paste your keys into these two lines near the top of the code, then save:

```
const API_KEY  = "paste-your-finnhub-key-here";
const HIST_KEY = "paste-your-twelvedata-key-here";  # optional, for history
```

> **SECURITY**
> Hardcoding the key is fine for your own personal copy. Do NOT do this on a public website - anyone could read it. For a public app, the key lives on a backend (Part 5).

Using it

- Type any company or ticker in the search box. Covered names (GOOGL, MSFT, META, AMZN) open the full written analysis; any other stock opens the live quantitative view.
- With the Finnhub key on, the search autocompletes across every listed stock.

Part 3 - Watchlist, portfolio summary, history, news and alerts

Your Watchlist home screen

The app opens on your Watchlist - a clean table of every stock you track. Each row shows price, today's move, where it sits in its 52-week range, its support level, and an **Entry watch** badge.

- Green badge / row = trading in its support zone (the area some investors watch for entry).
- Yellow = approaching support; red = near its 52-week high; grey = mid-range.
- Tap any row to open the full breakdown for that stock.
- Edit the HOLDINGS line near the top of the HTML to set your own list permanently.

Portfolio summary

Above the table, a summary strip shows your watchlist at a glance: average move today, how many names are advancing vs declining, how many are near support, and the day's leader and laggard. To turn this into a true money value (total holdings worth and dollar profit/loss), add a share-count per holding - a small extension you can do in the file or store in the backend (Part 5).

Latest news

Each stock shows its recent headlines (source, date, and a short snippet) pulled from Finnhub, linking out to the full article. It uses the same Finnhub key - no extra setup.

The price-history chart

Each stock page shows a one-year daily closing-price line so you can see the real trend behind the support and resistance zones. It loads when your Twelve Data key is on.

Price alerts (your 'don't miss it' monitor)

1. Make sure Live data is on (alerts need the Finnhub key to check prices).
2. On any stock, tap **Alert me near support** - it pre-fills the support price. Or open the **Alerts** panel and add a ticker + your own price.
3. The app checks your watched stocks **every minute while the page is open** and shows how far each is from its target.
4. When a stock reaches your price you get an on-screen pop-up and (if you allow it) a browser notification.

READ THIS

Browser alerts only run **while the page is open**. For alerts that fire even when the app is closed (email/SMS), use the backend in Part 5 and Part 6. The browser version is great for a tab kept open; the backend version is true hands-off monitoring.

MINDSET

Alerts and the green 'entry watch' badge are a heads-up to LOOK, not a buy signal. A stock can sit in its support zone because something is genuinely wrong - always check the news and analysis, and use a plan.

Part 4 - How the whole thing fits together

Right now your browser talks straight to the data companies. That's great for personal use, but a real app puts a small 'middle layer' in between. Here's the picture.

Piece	What it does	Suggested tool
Frontend	The dashboard people see and click	Your HTML file, or React / React Native
Data providers	Send prices, fundamentals, history, news	Finnhub, Twelve Data, Marketaux
Backend	Hides your keys, caches data, runs logic	Supabase Edge Functions / Cloudflare Workers
Database	Stores snapshots, watchlists, alerts	Supabase (Postgres)
Scheduler	Refreshes data on a timer (daily etc.)	Supabase Cron / Cloudflare Cron / GitHub Actions

The flow becomes: **scheduler pulls data -> stores it in the database -> your dashboard reads from your own database.** That makes the app fast, keeps your keys secret, and means you never hit rate limits from many users.

Part 5 - Turn it into a real, daily-updating app

A practical path using Supabase (which you already use). You can ask Claude Code to scaffold each step.

> **FILES INCLUDED**
> Two ready-made files come with this guide: **supabase_schema.sql** (all your tables, pre-seeded with your watchlist) and **refresh-function.ts** (the scheduled job that pulls prices, news and history and checks your alerts). Hand both to Claude Code and ask it to set them up - the steps below explain what each does.

Step 1 - Host the dashboard

- Free options: Netlify, Vercel, Cloudflare Pages, or GitHub Pages. Drag-and-drop your HTML file or connect a repo.
- Now you can open MoneyMochi from your phone, and add it to your home screen.

Step 2 - Create a Supabase project + tables

A simple starting schema:

```
# table: stocks      - one row per ticker you track
ticker text primary key, name text, sector text, updated_at timestamptz
# table: quotes      - latest price snapshot per ticker
ticker text, price numeric, change_pct numeric, pe numeric,
market_cap numeric, week52_high numeric, week52_low numeric, as_of timestamptz
# table: prices_daily - history for the chart
ticker text, date date, close numeric,  primary key (ticker, date)
# table: watches     - your price alerts
id uuid, user_id uuid, ticker text, target numeric, mode text, triggered bool
# table: news        - cached headlines
id uuid, ticker text, headline text, url text, source text, published_at timestamptz
```

Step 3 - A scheduled refresh function

Create a Supabase Edge Function that fetches data and writes it to your tables, then schedule it (e.g. a few times a day). Your API keys live here as secrets - never in the frontend.

```
// supabase/functions/refresh/index.ts  (sketch)
const FINNHUB = Deno.env.get('FINNHUB_KEY');
for (const ticker of tickers) {
  const q = await fetch(`https://finnhub.io/api/v1/quote?symbol=${ticker}&token;=${FIN
NHUB}`)
             .then(r => r.json());
  await supabase.from('quotes').upsert({
    ticker, price: q.c, change_pct: q.dp, as_of: new Date().toISOString()
  });
}
// then schedule it with a cron expression, e.g. every 30 min during market hours
```

THE KEY CHANGE

Your dashboard then reads from Supabase instead of calling Finnhub directly. Swap the fetch URLs in the HTML from finnhub.io to your Supabase REST endpoint. Claude Code can do this refactor for you.

Step 4 - Add news

- Use Finnhub company-news or a provider like Marketaux. Pull headlines in the same scheduled function and store them in the news table.
- Your frontend shows the latest few per stock - now you have the 'daily updated news' you wanted.

Part 6 - Notifications that work when the app is closed

The browser alerts in Part 3 only run while the page is open. To never miss a move, move the checking to your backend. Three levels, easiest first:

Method	Fires when app closed?	What you need
Browser notification	No (tab must be open)	Already built in - zero setup
Email alert	Yes	Backend cron + an email API (Resend, SendGrid)
SMS / WhatsApp alert	Yes	Backend cron + Twilio (small per-message cost)
Mobile push	Yes	A real mobile app (Expo) + push service

The recipe for hands-off alerts

1. Store each alert (ticker + target price) in the **watches** table.
2. A scheduled backend function checks current prices against every target (it already has your keys).
3. When price <= target, it sends an email or SMS and marks the alert as triggered so you aren't spammed.
4. Optionally reset the alert if the price climbs back well above the target.

> **GOOD NEWS**
> This is the same logic the in-browser version uses - you're just moving it to a server that's always on. A single Supabase scheduled function can handle all your alerts.

Part 7 - Can AI predict the market?

Short answer: no - not reliably, and you should be cautious of any app or person who claims it can. Here's the honest picture, because believing otherwise can cost real money.

Why prediction doesn't work

- Short-term prices are close to a 'random walk' - they react to brand-new information that, by definition, can't be known in advance.
- Markets are highly competitive. If a reliable signal existed, large funds with huge budgets would trade it away in seconds.
- AI models learn from the past. Markets constantly change, so patterns that worked before often stop working - and confident-looking forecasts are frequently just over-fitting.
- A model that's right 7 days running can still be very wrong on the 8th, which is the one that hurts.

> **RED FLAG**
> Be especially wary of any tool promising 'AI buy signals', guaranteed returns, or price targets. That's a red flag, not a feature. MoneyMochi deliberately shows zones and context, never 'buy now'.

What AI genuinely CAN do in your app

- Gather all the updated information in one place - prices, fundamentals, news, your watchlist - so you're not hopping between tabs.
- Summarise long filings and news into plain language, and answer your questions about a company.
- Generate balanced, educational analysis (business model, bull and bear case) from current data - refreshed on a schedule.
- Watch your chosen levels and alert you when something needs your attention.
- Spot and describe patterns or anomalies - while being honest that describing the past is not foretelling the future.

> **THE TAKEAWAY**
> So yes - you can absolutely have 'all the updated information in one app'. That's organise-and-inform, which AI is great at. Just keep it separate in your mind from 'predict-the-future', which nobody can do reliably. Use the app to be better informed, then decide with your own plan (and ideally dollar-cost averaging).

Part 8 - Making the analysis itself update

Today the written deep-dives, scores, and invalidation levels are fixed text (a June 2026 snapshot). The live numbers update, but the story doesn't. To make the analysis refresh with the market, have an AI model rewrite it from fresh data on a schedule.

1. In your scheduled backend function, gather the latest figures and a few recent headlines for the stock.
2. Send them to an AI model (for example the Anthropic Claude API) with a prompt like: 'Using these numbers and news, write a beginner-friendly business summary, bull/bear case, and a 1-10 score. Educational only - no buy/sell advice.'
3. Save the generated text in a table (e.g. analysis: ticker, body, score, generated_at).
4. Your dashboard shows the freshest version, with the date it was generated.

DO THIS

Always keep the 'educational, not advice' instruction in the prompt, and show the generation date so readers know how current it is. Have the model cite the data it used.

BONUS

This is also how you'd give every stock a written analysis - not just the four covered names - without hand-writing each one.

Part 9 - Costs and the licensing gotcha

Stage	Rough monthly cost
Personal dashboard (free tiers, delayed data)	$0
Hosting (Netlify/Vercel/Cloudflare free tier)	$0
Supabase free tier (small project)	$0
Paid data tier (more calls / real-time)	~$30-100+
Email/SMS alerts (Resend / Twilio)	a few dollars + usage
AI analysis (Claude API, scheduled)	usage-based, small at low volume

LICENSING - DO NOT SKIP

The big one most people miss: **data redistribution licensing**. Free and cheap data tiers usually allow PERSONAL use only. The moment you show live market data to OTHER users, most providers require a higher paid 'redistribution' licence, and real-time exchange-licensed data for a public product can get genuinely expensive (it involves agreements with the exchanges). For your own private dashboard you're completely fine. Before you let others use it, read each provider's redistribution terms.

Part 10 - Go-live checklist + glossary

Before you share it

- Keys are on the backend as secrets, never in public frontend code.
- A clear 'educational only, not financial advice' notice is visible (yours already has one).
- Data shows an 'as of' time so nobody mistakes delayed data for live.
- You've checked the redistribution terms of every data provider.
- Alerts can't spam (they trigger once, then reset sensibly).
- No prediction or guaranteed-return claims anywhere - the app informs, it doesn't foretell.
- You never store anyone's passwords or sensitive personal data.

Mini glossary

API key	A password that lets your app request data from a provider.
Endpoint	A specific URL you call to get one kind of data (quote, profile, history).
Rate limit	The max number of requests allowed per minute/day on your plan.
Caching	Saving a fetched result so you don't re-request it constantly.
Cron / scheduler	A timer that runs your code automatically (e.g. daily).
Backend	Server-side code that hides keys and does work the browser shouldn't.
Support / resistance	Price zones where buyers / sellers have tended to appear.
Redistribution licence	Permission to show a provider's data to other people.

Useful links

- Finnhub (data + search): finnhub.io
- Twelve Data (history): twelvedata.com
- Supabase (backend + DB + cron): supabase.com
- Hosting: netlify.com, vercel.com, pages.cloudflare.com
- News: marketaux.com, or Finnhub company-news

> **FINAL WORD**
> You've got this. Start small: run the file, add your keys, set one alert. Then add the backend one piece at a time. Every step here is something Claude Code can help you build.